

DOCUMENTATION D'ERREUR

1. Premièrement nous avons d'abord détecter des erreurs de types silencieuses. Car si une classe est créer elle doit etre initialiser avec son constructeur.

Voici le type d'erreur silencieuse : elle est soulignée en bleu

```
class MonApplication extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {
```

Alors pour faire disparaître cette erreur qui d'ailleurs n'empêche pas l'exécution de l'application, il faut initialiser la classe par son constructeur comme le montre l'image suivant. Et on voit directement la disparition du trait en **bleu**

```
class MonApplication extends StatelessWidget {  
  const MonApplication({super.key});  
  @override
```

Pour se faire on va faire ça pour toutes les classes existantes.

NB : cette erreur n'empêche pas l'exécution de l'application mais il est bon de corriger afin de garantir la performance et optimiser l'application.

2. En second lieu nous avons trouver un problème qui empêche l'exécution de l'application car le widget **padding** ne possède pas de propriété **mainAxisAlignment** et **crossAxisAlignment**. C'est le widget **column** qui possède ces propriétés.

Voici l'erreur

```
body: Padding(  
  padding: EdgeInsets.all(16.0),  
  mainAxisAlignment: MainAxisAlignment.start,  
  crossAxisAlignment: CrossAxisAlignment.stretch,  
  children: <Widget>[  
    Text(  
      'Calculer la moyenne de la classe',  
    ),  
  ],  
)
```

Alors pour corriger cette erreur nous allons inclure à l'intérieur du widget **padding** le widget **column** comme son enfant. Et voici l'image après correction

```
body: Padding(  
  padding: EdgeInsets.all(16.0),  
  child: Column(  
    mainAxisAlignment: MainAxisAlignment.start,  
    crossAxisAlignment: CrossAxisAlignment.stretch,  
    children: <Widget>[  
      Text(  
        'Calculer la moyenne de la classe',  
      ),  
    ],  
  ),  
)
```

3. Tertiaire nous avons rencontré un problème de syntaxe à partir de **elevetedButton**, voici l'image qui accompagne, et cette erreur est encadrer par un rectangle **jaune**

```
ElevatedButton(  
  child: Text('Calculer la moyenne de la classe'),  
  onPressed: () {  
    final average = calculateMoyenne(etudiants);  
    moyenneAlertDialog(context, average); // Afficher l'alerte  
  },  
)
```

On va remplacer cette caractère par la bonne et voici le résultat :

```
ElevatedButton(  
  child: Text('Calculer la moyenne de la classe'),  
  onPressed: () {  
    final average = calculateMoyenne(etudiants);  
    moyenneAlertDialog(context, average); // Afficher l'alerte  
  },  
)
```

4. En quatrieme position nous avons un probleme d'execution a partir d'ici :

```
total += etudiant.moyenne as int; total = 0.0
```

Donc nous allons enlever **as int**

```
total += etudiant.moyenne ;
```

Total est de type double et l'expression **as** ne convertit pas il vérifie juste le type.

5. Et en dernière position une erreur qui va bugger l'application car la variable total est déclarée à l'intérieur du boucle donc à chaque fois il se met à zéro, et pour corriger cette erreur il faut juste enlever la variable total à l'intérieur de la boucle.

Voici l'image de l'erreur en jaune

```
double calculateMoyenne(List<Etudiant> etudiants) {  
  double total = 0.0;  
  for (var etudiant in etudiants) {  
    var total = 0;  
    total += etudiant.moyenne as int;  
  }  
}
```

Après correction, on obtient ça :

```
double calculateMoyenne(List<Etudiant> etudiants) {  
  double total = 0.0;  
  for (var etudiant in etudiants) {  
    total += etudiant.moyenne as int;  
  }  
}
```

Et évidemment il faut corriger le nom de l'application dans le fichier **widget_test.dart**.

Comme nous montre l'erreur ici.

Nous allons dans le dossier test et dans le fichier **widget_test.dart** on a cette erreur

```
testWidgets('Counter increments smoke test', (WidgetTester tester) async {  
  // Build our app and trigger a frame.  
  await tester.pumpWidget(const MyApp());  
})
```

Alors pour ce fait, nous allons corriger ça, par la classe qui est le point d'entrée de mon application comme suit :

```
void main() {  
  Run | Debug  
  testWidgets('Counter increments smoke test', (WidgetTester tester) async {  
    // Build our app and trigger a frame.  
    await tester.pumpWidget(const MonApplication());  
  })  
}
```

B- TEST UNITAIRE

Pour ce fait on va tester la fonction calculateMoyenne

```
To https://github.com/HackIsHustler/qualite_logiciel.git  
4937a01..2d6f594 main -> main  
PS Z:\DOSSIER FLUTTER\formation_flutter_intermediaire oif\Niveau avance\semaine 5\qualite_mobile_s5> flutter test test/calcul_test.dart  
00:08 +1: All tests passed!  
PS Z:\DOSSIER FLUTTER\formation_flutter_intermediaire oif\Niveau avance\semaine 5\qualite_mobile_s5> █
```

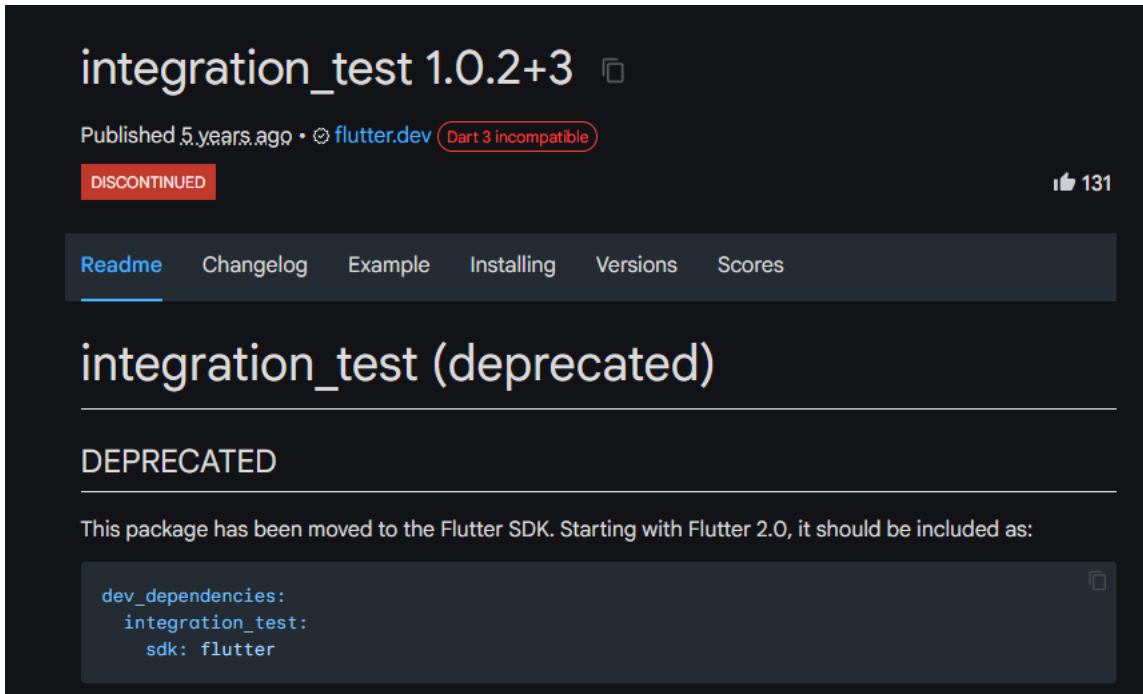
D'après la capture d'écran, le test s'est passé avec succès.

C- TEST DE WIDGET

```
To run this test again: C:\flutter\bin\cache\dart-sdk\bin\dart.exe test Z:\DOSSIER FLUTTER\formation_flutter_intermediaire oif\Niveau avance\semaine 5\qualite_mobile_s5\test/widget_tests.dart -p vm --plain-name "le bouton ""Calculer la moyenne"" est présent"
00:04 +0 -1: Some tests failed.
PS Z:\DOSSIER FLUTTER\formation_flutter_intermediaire oif\Niveau avance\semaine 5\qualite_mobile_s5>
```

Voici le test sur le widget pour verifier la presence du bouton dans le code.

D- TEST D'INTEGRATION

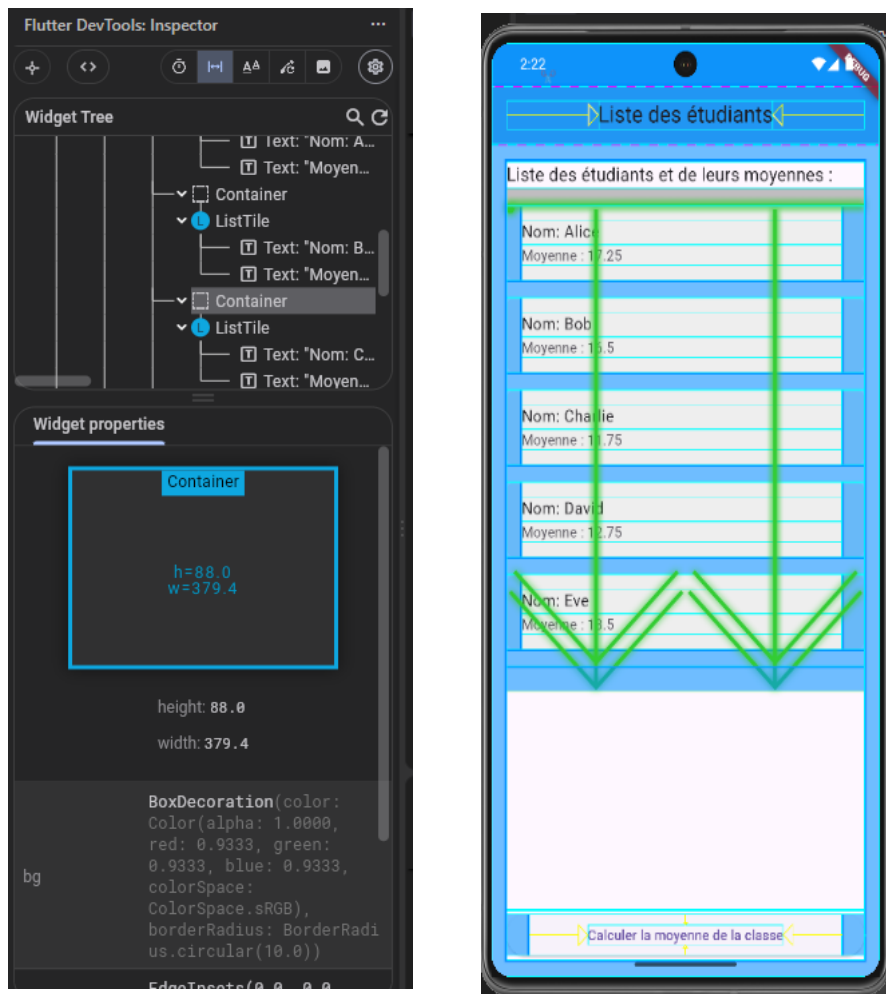


Je voulais intégrer le test d'intégration et le package énoncer dans le cours est révolue

```
PS Z:\DOSSIER FLUTTER\formation_flutter_intermediaire oif\Niveau avance\semaine 5\qualite_mobile_s5> flutter pub add integration_test
Because qualite_mobile_s5 depends on integration_test any which doesn't support null safety, version solving failed.

The lower bound of "sdk: '>=2.2.2 <3.0.0'" must be 2.12.0 or higher to enable null safety.
For details, see https://dart.dev/null-safety
Failed to update packages.
PS Z:\DOSSIER FLUTTER\formation_flutter_intermediaire oif\Niveau avance\semaine 5\qualite_mobile_s5>
```

Pour le devtools, j'utilise directement l'outil intégrer dans mon vscode.



Pour ce qui est de proposition, j'aimerais ajouter un truc :

Un **ListTile** a besoin d'un widget **Material** juste au-dessus de lui pour dessiner correctement son effet d'encre (le petit effet visuel gris au tap) et son fond. Mais ici, c'est un **Container** avec un **BoxDecoration** coloré qui l'entoure directement — Flutter considère que ça peut **cacher** l'effet d'encre du **ListTile**, et lève une assertion (erreur) à ce sujet en mode debug/test.

Cette assertion se déclenche **5 fois** (une par étudiant dans ta liste), ce qui fait échouer le test entier .